

Protocolos TCP/UDP

Objetivo da Aula

Conhecer o funcionamento dos protocolos TCP e UDP, suas principais diferenças e aplicações.

Apresentação

No modelo TCP/IP, a camada de transporte usa dois protocolos: TCP, orientado à conexão; e UDP, não orientado à conexão. Podemos comparar o TCP com serviço de carta registrada do correio, em que o remetente recebe um aviso de que a carta foi entregue com sucesso no destino. Já o UDP é mais como uma carta simples, um datagrama, em que o correio não informa ao remetente se a correspondência chegou ou não. Nesta aula, vamos conhecer os principais detalhes e aplicações desses protocolos.

1. Protocolo TCP

O protocolo TCP é um protocolo da camada de transporte, do modelo TCP/IP, orientado à conexão, definido pela IETF na RFC 793, que, além de suportar as funções básicas de segmentação e remontagem de dados, fornece os serviços de estabelecimento da conexão, entrega confiável, controle de erro e controle de fluxo para as conversações entre os protocolos da camada de aplicação.

1.1. O Cabeçalho TCP

O cabeçalho TCP possui 20 bytes (160 bits) e encapsula a PDU da camada de aplicação. A figura 1 apresenta os campos em um cabeçalho TCP.

Figura 1: Cabeçalho TCP

0			16		31
Porta de Origem			Porta de destino		
Número de Sequencia					
Número de Confirmação					
Tam. cabeçalho	Resevado	Controle	Tamanho da Janela		
Checksum			Urgente		
Opções					
PDU camamada de Aplicação					

Fonte: Elaboração própria.

O quadro 1 apresenta os principais campos do cabeçalho TCP:

Quadro 1: Principais campos do cabeçalho TCP

Campo de cabeçalho TCP	Descrição
Porta de origem (16 bits)	Usado para identificar o aplicativo de origem por número de porta.
Porta de destino (16 bits)	Usado para identificar o aplicativo de destino por porta número.
Número de Sequência (32 bits)	Usado para fins de remontagem de dados.
Número de Confirmação (32 bits)	Usado para indicar que os dados foram recebidos e o próximo byte esperado da fonte.
Tamanho do cabeçalho (4 bits)	Conhecido como "offset", indica o comprimento do cabeçalho do segmento TCP.
Reservado (6 bits)	Reservado para uso futuro.

Controle (6 bits)	Inclui os códigos de bits, ou sinalizadores, que indicam a finalidade e função do segmento TCP. Os seis bits de controle sinalizadores são os seguintes: URG – Ativa o campo de Urgência; ACK – Indicador de confirmação usado no estabelecimento de conexão e encerramento de sessão; PSH – Função Push; RST – Redefine a conexão quando ocorrer um erro ou tempo limite; SYN – Sincroniza números de sequência usados no estabelecimento de conexão; FIN – Não há mais dados do remetente e usados no encerramento da sessão.
Tamanho da janela (16 bits)	Usado para indicar o número de bytes que podem ser aceitos de uma só vez.
Checksum (16 bits)	Usado para verificação de erros do cabeçalho e dos dados do segmento.
Urgente (16 bits)	Usado para indicar o tipo de urgência dos dados.

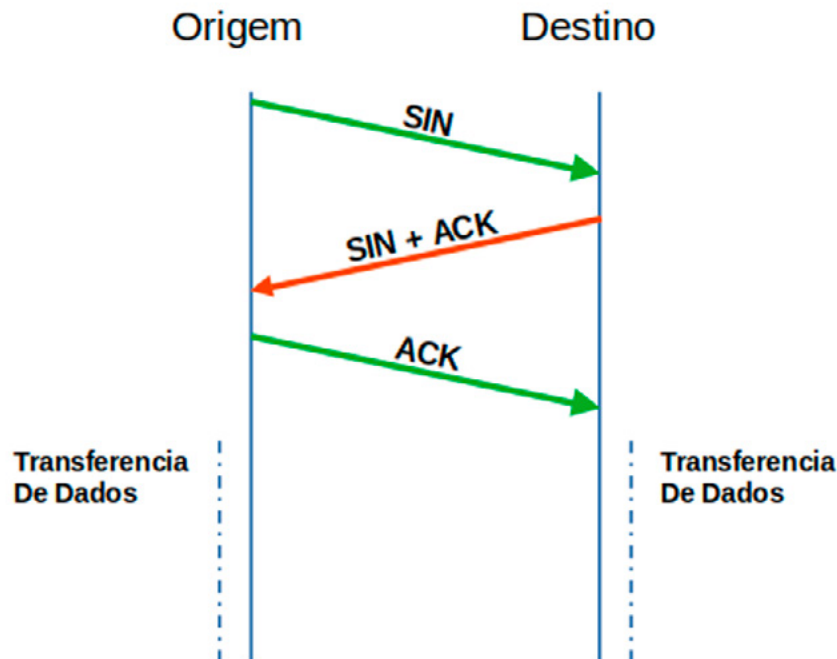
Fonte: Elaboração própria.

2. Funcionalidades de Protocolo TCP

2.1. Estabelecimento de Conexão TCP

Assim como nas conversas entre nós humanos, quando duas aplicações trocam dados é preciso que antes do diálogo seja estabelecida uma conexão, e para isso é feito um handshake (aperto de mãos). Nas conexões TCP, o host cliente estabelece a conexão com o servidor usando o processo de handshake de três vias (Three Way Handshake). A figura 2 apresenta esquematicamente esse processo:

Figura 2: Abertura de uma conexão TCP



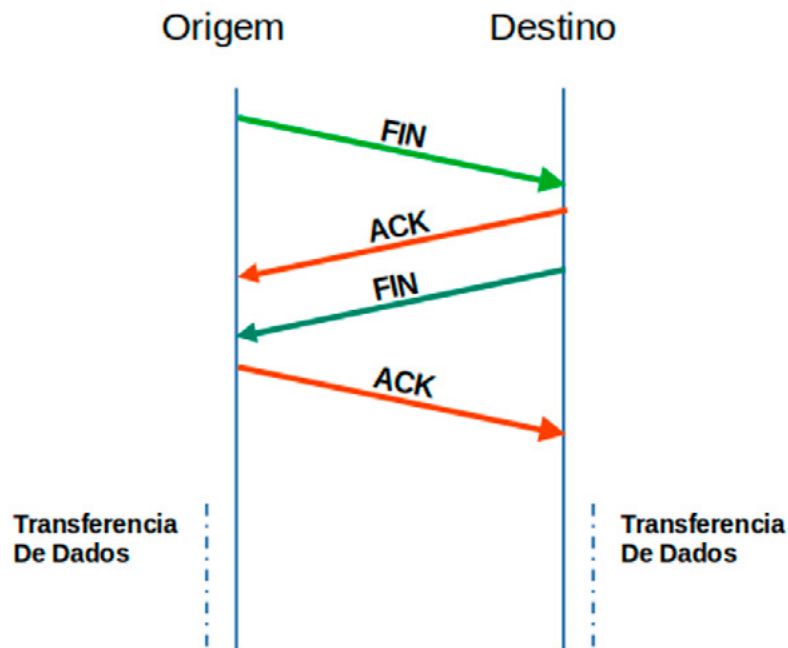
Fonte: Elaboração própria.

O cliente requisita uma sessão de comunicação cliente-servidor com o servidor enviando um segmento com o flag de controle SYN ativado, o servidor confirma a sessão de comunicação cliente-servidor e requisita uma sessão de comunicação de servidor-cliente com um SYN + ACK(Acknowledgment). O cliente iniciador confirma a sessão de comunicação de servidor-cliente, enviando um ACK. O handshake de três vias permite a validação mútua, garantindo que ambos se encontram disponíveis para comunicação.

2.2. Encerramento da Conexão

Para terminar uma sessão TCP, utiliza-se um handshake duplo, que consiste em um segmento FIN e um segmento ACK (Acknowledgment) para cada um dos participantes. Portanto, para terminar uma conversa única, quatro trocas são necessárias, que são apresentadas na figura 3:

Figura 3: Encerramento da conexão TCP



Fonte: Elaboração própria.

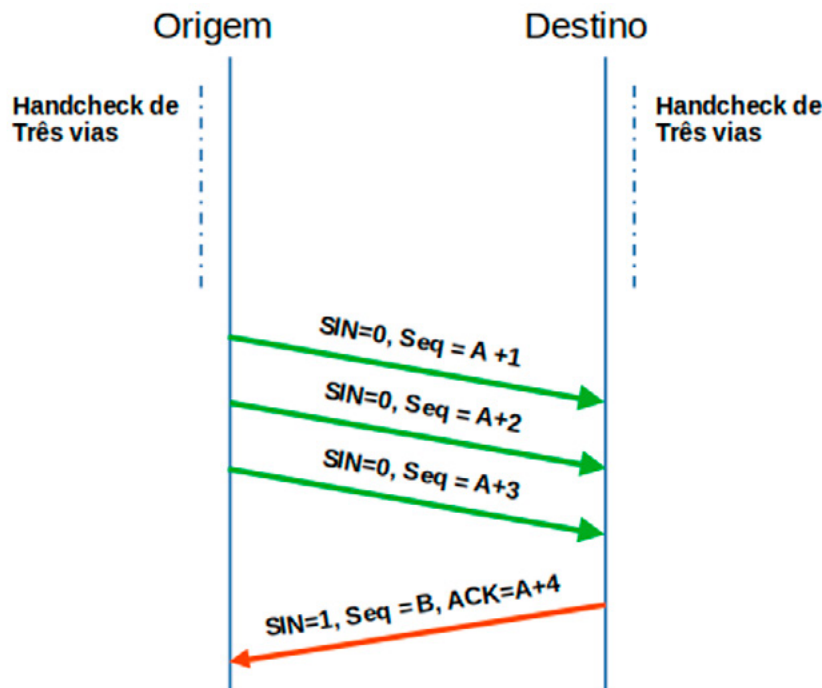
Tanto o cliente quanto o servidor podem solicitar o encerramento enviando um segmento com o flag de controle Finish (FIN) ligado no cabeçalho do segmento. Aquele que recebe a solicitação responde com um ACK para confirmar o recebimento de FIN e encerrar a seu lado da conexão. Em seguida, envia um FIN ao seu par para encerrar a sessão do outro lado, que responde com um ACK para reconhecer o FIN e encerrar sua própria conexão.

2.3. Números de Sequência

O TCP é um protocolo confiável porque ele reenvia segmentos descartados e os numera para indicar sua ordem correta antes da entrega, já que estes podem não chegar ao seu destino, ou às vezes chegar fora de ordem, devido a comutação de pacotes da camada 3. Para que a mensagem original seja entendida pelo destinatário, todos os dados devem ser recebidos e remontados na ordem original. Os números de sequência são atribuídos no cabeçalho de cada pacote para alcançar esse objetivo.

Após o estabelecimento da conexão, um número de sequência inicial (ISN – Initial Sequence Number) é definido, indicando o valor inicial para os bytes que são transmitidos ao aplicativo receptor. A figura 4 apresenta um esquema para incremento do ISN:

Figura 4: Incremento dos ISN



Fonte: Elaboração própria.

À medida que os dados são transmitidos durante a sessão, o número de sequência "A" é incrementado do número de bytes que serão transmitidos, gerando, para cada segmento, um número único para aquela conexão, permitindo que cada um seja identificado e confirmado de forma única. Segmentos perdidos podem, com isso, ser identificados.



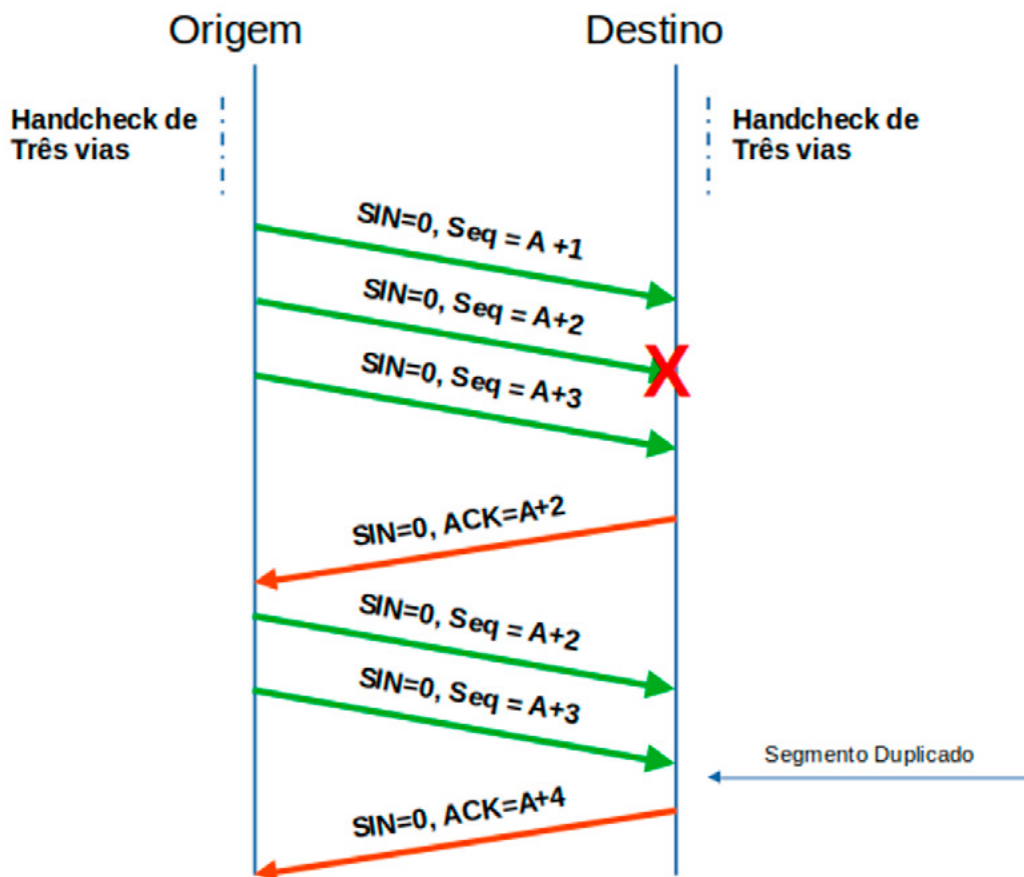
Destaque

O ISN começa com um número aleatório, para impedir tipos de ataques maliciosos.

2.4. Controle de Erro

O número de sequência (SEQ) e o número de confirmação (ACK) são usados para confirmar o recebimento dos segmentos. O número SEQ identifica o byte que está sendo transmitido, e o número de confirmação (ACK) enviado de volta à origem indica o próximo byte que o destino espera receber. Isso é chamado de confirmação antecipatória. Inicialmente, o TCP só podia reconhecer o próximo byte esperado. Por exemplo, a figura 5 apresenta esquematicamente um processo de transmissão e confirmação de segmentos utilizando ACK.

Figura 5: Transmissão e confirmação de segmentos TCP utilizando ACK

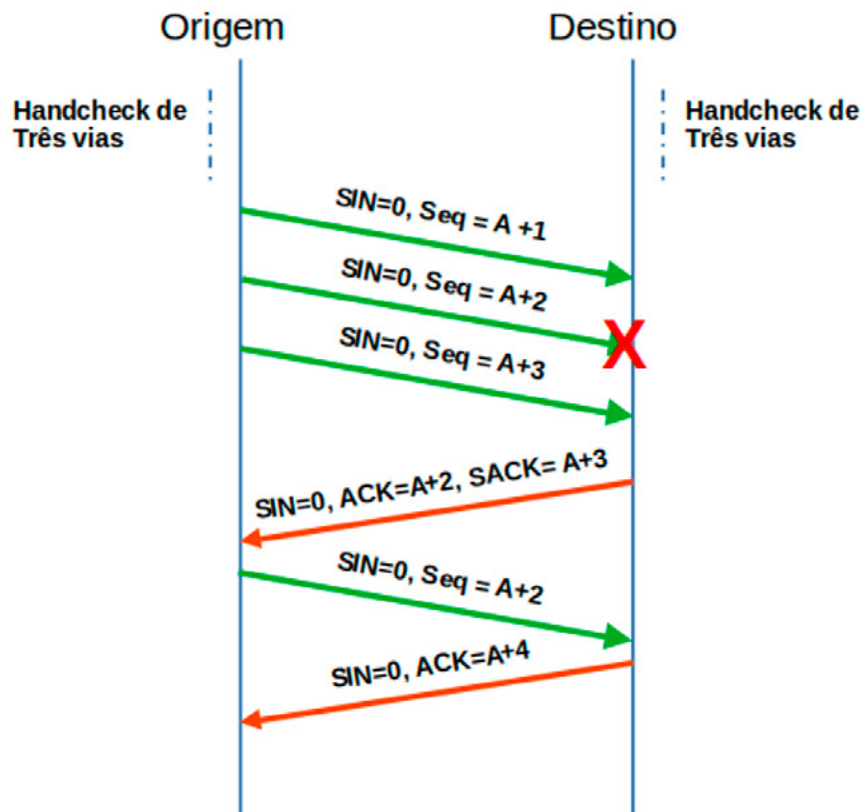


Fonte: Elaboração própria.

O host origem envia os segmentos de $A+1$ a $A+3$. Se todos os segmentos chegarem, exceto o segmento $A+2$, o host de destino responderia com um ACK indicando que o próximo segmento esperado é o $A+3$. Como o host de origem não sabe se outros segmentos chegaram, reenviaria os segmentos $A+2$ e $A+3$. Isso acarreta a transmissão de segmentos duplicados, ocasionando desperdício de banda e problemas de congestionamento.

Atualmente, os sistemas operacionais utilizam um recurso chamado reconhecimento seletivo (SACK), negociado durante o handshake de três vias. Se ambos os hosts suportarem SACK, o receptor pode reconhecer explicitamente quais segmentos (bytes) foram recebidos e quais não, e o emissor só precisa transmitir os segmentos ausentes. A figura 6 apresenta este esquema de transmissão utilizando SACK.

Figura 6: Transmissão e confirmação de segmentos TCP utilizando SACK



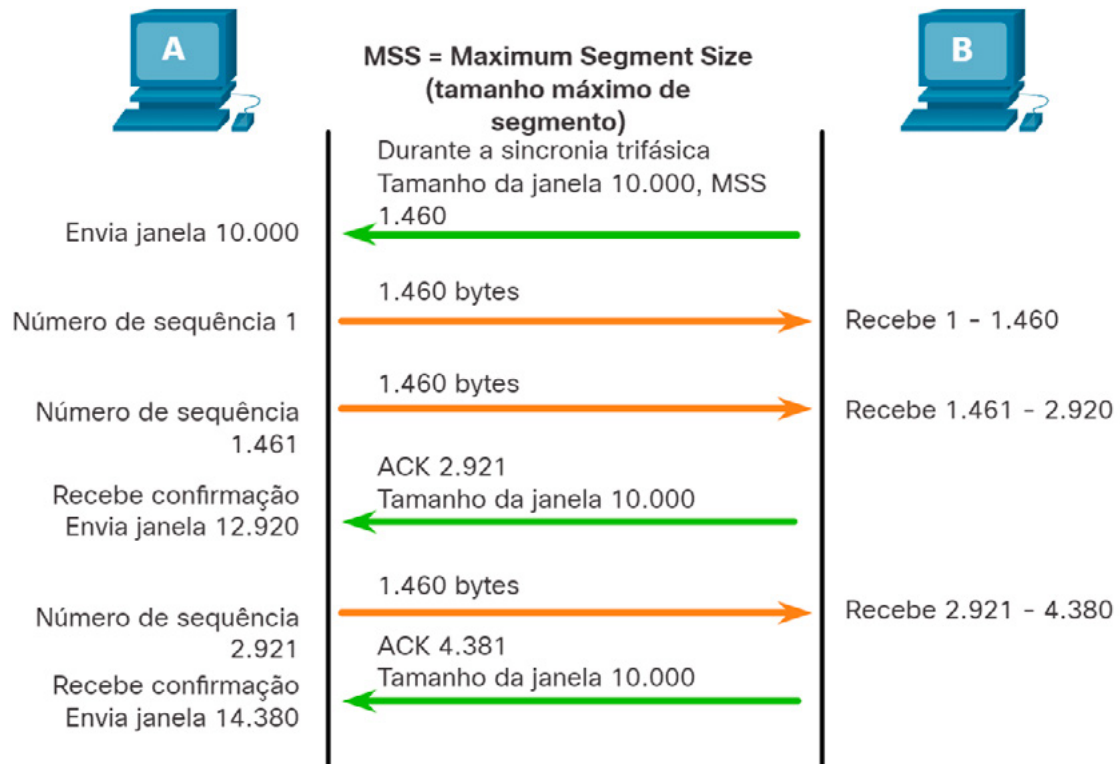
Fonte: Elaboração própria.

Aqui o host origem envia os segmentos de A+1 a A+3 para o host de destino. Se todos os segmentos chegarem, exceto o segmento A+2, o host de destino responderia com confirmação, especificando que o próximo segmento esperado é o segmento A+2 ($ACK=A+2$) e que recebeu seletivamente o segmento A+3 ($SACK=A+3$). O host de origem então reenviaria apenas o segmento A+2. Isso evita segmentos duplicados e desperdício de banda.

2.5. Controle de fluxo

O TCP utiliza o campo tamanho da janela, que determina o número de bytes que podem ser enviados antes de esperar uma confirmação e o tamanho máximo da janela (MSS – Maximum segment Size), trocado no campo “opções > fornecer controle de fluxo”, regulando a quantidade de dados que o destino pode receber e processar de forma confiável. O número de confirmação (ACK) corresponde ao próximo byte esperado. A figura 7 mostra o controle de fluxo entre dois processos utilizando o protocolo TCP.

Figura 7: Controle de fluxo TCP



Fonte: Network Academy.

Neste exemplo, o tamanho da janela inicial do PC B para a sessão TCP é de 10.000 bytes e o MSS é de 1460 bytes. Tomando o primeiro byte igual a 1, o último byte que PC A pode enviar sem receber uma confirmação é o byte 10.000. Isso é conhecido como janela de envio do PC A. O tamanho da janela pode ser fixo, isto é, não pode ser alterado naquela conexão ou deslizante, que pode ser alterado ao longo dela. Nesse caso, o tamanho da janela é incluído em todos os segmentos TCP. A vantagem de janelas deslizantes é que permitem que o emissor transmita continuamente segmentos, desde que o receptor esteja reconhecendo segmentos anteriores.

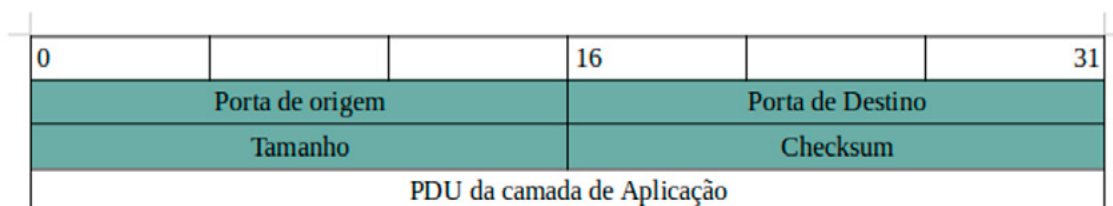
3. Protocolo UDP

O protocolo UDP, definido pela RFC 768, é um protocolo não orientado à conexão, leve, que oferece a mesma segmentação de dados e remontagem que o TCP, porém não faz estabelecimento de conexão e é sem confiabilidade, o que significa que pacotes perdidos não são retransmitidos e não faz controle de fluxo.

3.1. Datagrama UDP

As PDUs de comunicação no UDP são chamadas de datagramas e enviadas como o melhor esforço pelo protocolo da camada de transporte. Seu cabeçalho é muito mais simples do que o do TCP, só tem quatro campos e requer 8 bytes (ou seja, 64 bits) apenas. A figura 8 apresenta a estrutura de uma datagrama UDP:

Figura 8: Datagrama UDP



Fonte: Elaboração própria.

O quadro 2 descreve os campos do cabeçalho UDP:

Quadro 2: Datagrama UDP

Campo do Cabeçalho UDP	Descrição
Porta de origem (16 bits)	Usado para identificar a porta de origem.
Porta de destino (16 bits)	Usado para identificar a porta de destino.
Tamanho (16 bits)	Indica o comprimento do cabeçalho do datagrama UDP.
Checksum (16 bits)	Usado para detecção de erros do cabeçalho e dos dados do datagrama.

Fonte: Elaboração própria.

Por ser muito mais simples que o TCP, ele é muito mais rápido, sendo ideal para as aplicações que precisam que os dados fluam rapidamente e podem tolerar alguma perda de dados com efeito mínimo no resultado final. Há três tipos de aplicações que são mais adequadas para o UDP:

- **Aplicativos de vídeo e multimídia ao vivo** – esses aplicativos podem tolerar a perda de dados, mas requerem pouco ou nenhum atraso. Os exemplos incluem VoIP e transmissão de vídeo ao vivo.
- **Solicitações simples e aplicativos de resposta** – aplicativos com transações simples, em que um host envia uma solicitação e pode ou não receber uma resposta. Os exemplos incluem DNS e DHCP.
- **Aplicativos que lidam com a confiabilidade** – em comunicações unidirecionais em que o controle de fluxo, a detecção de erros não é necessária ou pode ser gerenciada pela camada de aplicação. Os exemplos incluem SNMP e TFTP.

Considerações Finais da Aula

A camada de transporte é responsável pela comunicação lógica entre aplicativos. Os protocolos dessa camada definem como transferir mensagens entre hosts e é responsável por gerenciar os requisitos de confiabilidade de uma conversa. No modelo TCP/IP, a camada de transporte inclui os protocolos TCP e UDP, sendo o TCP é stateful, confiável, reconhece dados, reenvia dados perdidos e entrega dados em ordem sequencial; este protocolo é utilizado por aplicações onde os dados não podem ser transferidos com erros, como para correio eletrônico e aplicações Web. Já o UDP é sem estado, rápido, tem baixa sobrecarga, não requer confirmação, não reenvia dados perdidos e fornece dados na ordem em que chegam, utilizado principalmente por aplicações que necessitem de velocidade e baixo atraso, como voz e vídeo, por exemplo.

Material Complementar

**Artigo**

O que é UDP e TCP? Entenda quais as diferenças e como funciona cada Protocolo

2023, Yan Orestes

No modelo TCP/IP, os protocolos da camada de transporte são o TCP e o UDP, cada um com suas particularidades. Para saber mais sobre a diferença entre eles, leia o artigo indicado.

Link para acesso: https://www.alura.com.br/artigos/quais-as-diferencas-entre-o-tcp-e-o-udp?utm_term=&utm_campaign=%5BSearch%5D+%5BPerformance%5D+--+Dynamic+Search+Ads+--+Artigos+e+Conte%C3%BAdos&utm_source=adwords&utm_medium=ppc&hsa_acc=7964138385&hsa_cam=11384329873&hsa_grp=111087461203&hsa_ad=662261334153&hsa_src=s&hsa_tgt=dsa-843358956400&hsa_kw=&hsa_mt=&hsa_net=adwords&hsa_ver=3&gclid=Cj0KCQjwi7GnBhDXARIsAFLvH-4kjLdlIso59pZRDj6eQT3TNSjqkqmqjgNPxkUceaBJKtKLiMPkGqTPQaAis_EALw_wcB.

Referências

BARRETO, Jeanine dos Santos *et al.* **Fundamentos de segurança da informação**. Porto Alegre: SAGAH, 2018.

COMER, Douglas E. **Redes de computadores e internet**. 6. ed. Porto Alegre: Bookman, 2016.

FOROUZAN, Behrouz A.; MOSHARRAF, Firouz. **Redes de computadores**: uma abordagem top-down. 1. ed. Porto Alegre: AMGH, 2013.

LACERDA, Paulo Sérgio Pádua de *et al.* **Projeto de redes de computadores**. Porto Alegre: SAGAH, 2022.

MAIA, Luiz Paulo. **Arquitetura de redes de computadores**. 2. ed. Rio de Janeiro: LTC, 2013.

MORAES, Alexandre Fernandes de. **Segurança em redes**: fundamentos. São Paulo: Erica, 2010.

NETWORK ACADEMY. **Introduction to Networks (CCNAv7)**. Disponível em: <https://www.netacad.com>. Acesso em: 3 jun. 2023.

SOUSA, Lindeberg Barros de. **Redes de computadores**: guia total. 1. ed. São Paulo: Erica, 2009.

_____. **Administração de redes locais**. 2. ed. São Paulo: Erica, 2020.